

# 教育本体构建系统模块接口设计

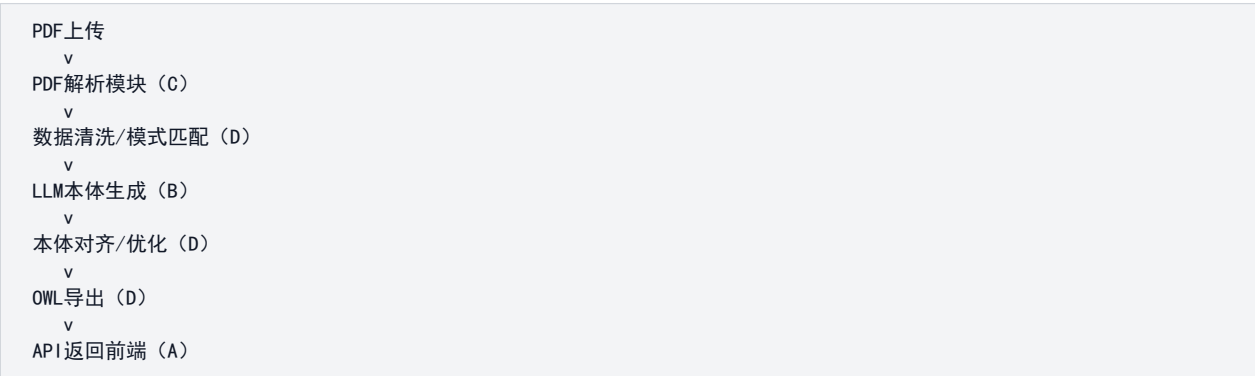
先讲团队分工与模块功能，再讲统一数据流、state和具体接口

## 一、ABCD负责模块与功能总结

本项目按后端处理流程拆分为四类职责：A负责API入口与前后端交互，B负责大模型本体生成，C负责PDF内容解析，D负责字段匹配、本体优化和OWL导出。各模块之间不直接改彼此数据结构，只通过统一 state 传递结果。

| 成员 | 负责模块            | 模块功能说明                                                                      |
|----|-----------------|-----------------------------------------------------------------------------|
| A  | API层            | 负责系统对外接口，包括PDF上传、触发本体生成、导出OWL下载链接。A的模块连接前端和后端流程，保证接口返回格式稳定。                 |
| B  | 本体生成模块          | 负责核心智能生成能力，将清洗后的结构化数据组织成 classes、properties、relations 等本体JSON，并封装LLM调用与提示词。 |
| C  | PDF解析模块         | 负责读取上传的PDF文件，提取正文和表格内容，并进行基础文本清理，输出统一的 raw_text 供后续模块使用。                    |
| D  | 模式匹配、本体对齐、OWL导出 | 负责把原始文本匹配为标准字段，对生成的本体做去重和实体合并，并最终导出OWL文件。                                   |

## 二、整体数据流（总架构）



## 三、统一数据结构

所有模块都必须遵守这个 state，这是 LangGraph 的核心状态对象。

```
state = {
  "file_path": "",
  "raw_text": "",
  "clean_data": {},
  "matched_data": {},
  "ontology": {},
  "owl_file": ""
}
```

## 四、具体模块接口设计

### 1. PDF解析模块（C负责）

文件：modules/pdf\_parser.py

```
def extract_pdf(file_path: str) -> str:
    """
    输入: PDF路径
    输出: 原始文本
    """

def extract_tables(file_path: str) -> list:
    """提取表格数据"""

def clean_text(text: str) -> str:
    """清理PDF文本"""
```

输出标准: raw\_text: str

## 2. 模式匹配模块 (D负责)

文件: modules/schema\_matcher.py

```
def match_schema(raw_text: str) -> dict:
    """
    输入: PDF原始文本
    输出: 标准化字段数据
    """

def normalize_field(name: str) -> str:
    """字段标准化"""

def similarity(a: str, b: str) -> float:
    """计算相似度"""

{
    "student_id": "...",
    "course_name": "...",
}
```

## 3. 本体生成模块 (B负责, 核心)

文件: modules/ontology\_builder.py

```
def build_ontology(clean_data: dict) -> dict:
    """
    输入: 结构化数据
    输出: 本体结构 (JSON)
    """

def call_llm(prompt: str) -> str:
    """调用大模型生成本体"""

{
    "classes": [],
    "properties": [],
    "relations": []
}
```

## 4. 本体优化/对齐模块 (D负责)

文件: modules/ontology\_aligner.py

```
def align_ontology(ontology: dict) -> dict:
    """
    输入: 初始本体
    输出: 优化后的本体
    """

def remove_duplicates(data: dict) -> dict:
    """去重"""

def merge_entities(a, b):
    """合并实体"""
```

## 5. OWL导出模块 (D负责)

文件: modules/owl\_generator.py

```
def generate_owl(ontology: dict) -> str:
    """
    输入: 本体结构
    输出: OWL文件路径
    """

def write_owl_file(content: str, path: str):
    """写入OWL文件"""

owl_file = "output.owl"
```

## 五、API层统一接口（A负责）

```
POST /upload
输入: file: PDF
输出: {"file_path": "xxx.pdf"}

POST /generate
输入: {"file_path": ""}
输出: {"ontology": {}, "status": "success"}

GET /export
输出: {"download_url": "xxx.owl"}
```

## 六、LangGraph统一节点接口

```
def node(state):
    return state

extract_node(state)
match_node(state)
llm_node(state)
owl_node(state)

state["raw_text"]
state["clean_data"]
state["ontology"]
state["owl_file"]
```

## 七、团队协作规则

### 禁止

- 改别人的函数签名
- 修改 state 结构
- 随便改接口返回值

### 必须

- 只改自己模块
- 严格遵守输入/输出格式
- 所有模块通过 state 通信

## 八、最终架构（答辩用）

```
PDF上传
  v
pdf_parser.extract_pdf()
  v
schema_matcher.match_schema()
  v
ontology_builder.build_ontology()
  v
ontology_aligner.align_ontology()
  v
owl_generator.generate_owl()
  v
API返回前端
```

## 九、这一套设计的优势

- 模块清晰
- 接口统一
- 易扩展
- 有工程规范
- LangGraph天然适配